

# Sentiment Analysis on SST-2

## Intelligent Systems & Machine Learning Project

Uvesh Patel  
Supervisor: Prof. Dario Bruneo

June 9, 2025

## Contents

|          |                                      |           |
|----------|--------------------------------------|-----------|
| <b>1</b> | <b>Data Loading</b>                  | <b>2</b>  |
| 1.1      | Dataset Description . . . . .        | 2         |
| 1.2      | Implementation . . . . .             | 2         |
| 1.3      | Terminal Output . . . . .            | 3         |
| 1.4      | Remarks . . . . .                    | 3         |
| <b>2</b> | <b>Text Preprocessing</b>            | <b>4</b>  |
| 2.1      | Purpose . . . . .                    | 4         |
| 2.2      | Implementation . . . . .             | 4         |
| 2.3      | Terminal Output . . . . .            | 6         |
| 2.4      | Explanation . . . . .                | 7         |
| 2.5      | Key Outcomes . . . . .               | 7         |
| <b>3</b> | <b>Feature Engineering</b>           | <b>7</b>  |
| 3.1      | Purpose . . . . .                    | 7         |
| 3.2      | Implementation . . . . .             | 8         |
| 3.3      | Terminal Output . . . . .            | 9         |
| 3.4      | Technical Explanation . . . . .      | 10        |
| 3.5      | Key Achievements . . . . .           | 11        |
| <b>4</b> | <b>Model Analysis</b>                | <b>11</b> |
| 4.1      | Models and Hyperparameters . . . . . | 11        |
| 4.2      | Analysis Techniques . . . . .        | 12        |
| 4.3      | Results and Discussion . . . . .     | 12        |

# 1 Data Loading

## 1.1 Dataset Description

The SST-2 dataset (Stanford Sentiment Treebank – binary subset) consists of short movie-review sentences labeled as positive or negative. We load three splits:

- **Training set:** 67 349 examples
- **Validation set:** 872 examples
- **Test set:** 1 821 examples

Each CSV (`data/raw/`) has these columns:

`sentence` (string) – the raw review text

`label` (integer) – 1 = positive, 0 = negative

`idx` (integer) – unique identifier in the original SST-2

**Class balance (train):**

$$P(\text{positive}) = \frac{37\,569}{67\,349} \times 100\% \approx 55.78\%, \quad P(\text{negative}) = \frac{29\,780}{67\,349} \times 100\% \approx 44.22\%.$$

**Sentence-length (train):**

$$\text{min} = 1, \quad \text{max} = 52, \quad \text{mean} \approx 9.41, \quad \text{median} = 7 \text{ words.}$$

**Sample record:**

`idx:` 102

`label:` 1

`sentence:` "The film's thoughtful tone and strong performances make it a delight."

## 1.2 Implementation

Listing 1: `01_data_loading.py`: Download, inspect and save SST-2.

```
import os
import pandas as pd
from datasets import load_dataset
from utils import RAW_DATA_DIR, save_dataframe

def download_sst2():
    print("Downloading SST-2 dataset...")
    ds = load_dataset("nyu-mll/glue", "sst2")
    return (pd.DataFrame(ds["train"]),
```

```

        pd.DataFrame(ds["validation"]),
        pd.DataFrame(ds["test"]))

def main():
    train_df, val_df, test_df = download_sst2()
    os.makedirs(RAW_DATA_DIR, exist_ok=True)
    save_dataframe(train_df, os.path.join(RAW_DATA_DIR, "sst2_train.
        csv"))
    save_dataframe(val_df, os.path.join(RAW_DATA_DIR, "sst2_val.
        csv"))
    save_dataframe(test_df, os.path.join(RAW_DATA_DIR, "sst2_test.
        csv"))
    print("Dataset downloaded and saved successfully!")

if __name__ == "__main__":
    main()

```

### 1.3 Terminal Output

```

PS D:\Machine Learning project\src> python 01_data_loading.py
Installing missing dependencies: scikit-learn
All dependencies installed successfully!
Downloading NLTK resource: stopwords
Downloading NLTK resource: wordnet
Downloading SST-2 dataset...
Loading SST-2 dataset from Hugging Face...
DataFrame saved to D:\Machine Learning project\data\raw\sst2_train.csv
DataFrame saved to D:\Machine Learning project\data\raw\sst2_val.csv
DataFrame saved to D:\Machine Learning project\data\raw\sst2_test.csv
Dataset downloaded and saved successfully!
PS D:\Machine Learning project\src>

```

### 1.4 Remarks

- `load_dataset("nyu-mll/glue", "sst2")` automatically handles download, parsing, and caching to `~/.cache/huggingface/datasets/`.
- Saving raw CSVs under `data/raw/` ensures an unmodified snapshot for audit and reproducibility.
- NLTK resources (`stopwords`, `wordnet`) are pre-fetched here to guarantee downstream scripts run without interruption.
- We capture *class balance* and *sentence-length statistics* to inform model choice (e.g. expected input sizes for LSTMs or CNNs).
- Including `idx` allows precise tracing of any misclassified examples back to original data.

## 2 Text Preprocessing

### 2.1 Purpose

Transform raw sentences into clean, normalized text to reduce noise and vocabulary sparsity. This step prepares the data for reliable feature extraction (BoW, TF-IDF) by:

- Unifying letter case so that “Movie!” and “movie” are treated identically.
- Removing punctuation that does not contribute sentiment (e.g., commas, periods, quotation marks).
- Tokenizing into individual words for downstream vectorizers.
- Eliminating stop-words (e.g. “the”, “and”) which are high-frequency but carry little sentiment.
- Optionally performing lemmatization to collapse inflected forms (e.g. “running” → “run”).

### 2.2 Implementation

Listing 2: 02\_preprocessing.py: Normalization and cleaning logic.

```
import os, re, pandas as pd, nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
import argparse
from utils import RAW_DATA_DIR, PROCESSED_DATA_DIR, load_dataframe,
    save_dataframe

def download_nltk_resources():
    for res in ['punkt', 'stopwords', 'wordnet']:
        try:
            nltk.data.find(f'tokenizers/{res}')
        except LookupError:
            nltk.download(res, quiet=True)

def preprocess_text(text, remove_stopwords=True, lemmatize=False):
    # 1. Lowercase
    text = text.lower()
    # 2. Remove punctuation
    text = re.sub(r'^a-z0-9\s', ' ', text)
    # 3. Tokenize
    tokens = word_tokenize(text)
    # 4. Stop-word removal
    if remove_stopwords:
```

```

        sw = set(stopwords.words('english'))
        tokens = [t for t in tokens if t not in sw]
# 5. Lemmatization
if lemmatize:
    lem = WordNetLemmatizer()
    tokens = [lem.lemmatize(t) for t in tokens]
# 6. Rejoin tokens
return ' '.join(tokens)

def preprocess_dataset(df, remove_stopwords=True, lemmatize=False):
    print(f"Preprocessing {len(df)} examples "
          f"(stopwords={remove_stopwords}, lemmatize={lemmatize})")
    df = df.copy()
    df['processed_text'] = (
        df['sentence']
        .apply(lambda x: preprocess_text(x, remove_stopwords,
                                          lemmatize))
    )
    return df

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--no-stopwords', action='store_false',
                        dest='remove_stopwords',
                        help='Skip stop-word removal')
    parser.add_argument('--lemmatize', action='store_true',
                        help='Enable lemmatization')
    args = parser.parse_args()

    download_nltk_resources()

    # Load raw data
    train_df = load_dataframe(os.path.join(RAW_DATA_DIR, 'sst2_train.
.csv'))
    val_df = load_dataframe(os.path.join(RAW_DATA_DIR, 'sst2_val.
.csv'))
    test_df = load_dataframe(os.path.join(RAW_DATA_DIR, 'sst2_test.
.csv'))

    # Apply preprocessing
    proc_train = preprocess_dataset(train_df,
                                    remove_stopwords=args.
                                        remove_stopwords,
                                    lemmatize=args.lemmatize)
    proc_val = preprocess_dataset(val_df,
                                   remove_stopwords=args.
                                       remove_stopwords,

```

```

                                lemmatize=args.lemmatize)
proc_test = preprocess_dataset(test_df,
                                remove_stopwords=args.
                                    remove_stopwords,
                                lemmatize=args.lemmatize)

# Save processed data
os.makedirs(PROCESSED_DATA_DIR, exist_ok=True)
save_dataframe(proc_train,
                os.path.join(PROCESSED_DATA_DIR, '
                    sst2_train_processed.csv'))
save_dataframe(proc_val,
                os.path.join(PROCESSED_DATA_DIR, '
                    sst2_val_processed.csv'))
save_dataframe(proc_test,
                os.path.join(PROCESSED_DATA_DIR, '
                    sst2_test_processed.csv'))

print("Preprocessing completed successfully!")
print("\nSample transformations:")
for i in range(5):
    orig = proc_train.iloc[i]['sentence']
    proc = proc_train.iloc[i]['processed_text']
    print(f"Original : {orig}")
    print(f"Processed: {proc}\n")

if __name__ == '__main__':
    main()

```

## 2.3 Terminal Output

```

PS> python 02_preprocessing.py
Installing dependencies...
Downloading NLTK resource: stopwords
Downloading NLTK resource: wordnet
DataFrame loaded from data/raw/sst2_train.csv
DataFrame loaded from data/raw/sst2_val.csv
DataFrame loaded from data/raw/sst2_test.csv
Preprocessing 67349 examples (stopwords=True, lemmatize=False)
Preprocessing 872 examples (stopwords=True, lemmatize=False)
Preprocessing 1821 examples (stopwords=True, lemmatize=False)
DataFrame saved to data/processed/sst2_train_processed.csv
DataFrame saved to data/processed/sst2_val_processed.csv
DataFrame saved to data/processed/sst2_test_processed.csv
Preprocessing completed successfully!

```

Sample transformations:

Original : hide new secretions from the parental units

Processed: hide new secretions parental units

...

## 2.4 Explanation

1. **Lowercasing:** Converts all text to lowercase to treat “Cinema” and “cinema” identically, halving the effective vocabulary.
2. **Punctuation Removal:** Regular expression `[a-z and 0-9]` targets any character that is not a lowercase letter, digit, or whitespace, replacing it with a space.
3. **Tokenization:** `word_tokenize` splits on word boundaries and handles edge-cases (e.g. apostrophes in “don’t”).
4. **Stop-word Removal:** Filters out the top 179 English function words (NLTK’s list), which occur in virtually every sentence yet rarely indicate sentiment.
5. **Lemmatization:** *Optional* step using WordNet to map inflected forms (e.g. “running”, “ran”) back to their lemma (“run”), reducing feature-space sparsity.
6. **Resulting Benefits:**
  - Noise reduction—punctuation and filler words are eliminated.
  - Vocabulary size is significantly decreased (faster training and lower memory).
  - Preserves sentiment-bearing tokens (*e.g.* “excellent”, “terrible”) for feature extraction.

## 2.5 Key Outcomes

- Generated clean CSVs with `processed_text` for all splits.
- Confirmed via samples that no sentiment information was lost—only non-informative tokens were removed.
- Established foundations for effective Bag-of-Words and TF-IDF feature generation.

# 3 Feature Engineering

## 3.1 Purpose

Convert cleaned text into fixed-length numerical feature vectors so that standard machine-learning algorithms can be applied. We implement two complementary representations:

- **Bag-of-Words (BoW):** counts of the top  $V$  tokens in each document.

- **TF-IDF:**  $\text{TF} \times \text{IDF}$  weighting to emphasize rare but informative words.

These representations turn each sentence into a vector  $x \in R^V$ , enabling regression and classification models to operate on text.

## 3.2 Implementation

Listing 3: 03\_feature\_engineering.py: BoW and TF-IDF extraction

```
import os, pickle, pandas as pd
from sklearn.feature_extraction.text import CountVectorizer,
    TfidfVectorizer
from utils import PROCESSED_DATA_DIR, MODEL_DIR, load_dataframe,
    save_pickle

def create_bow_features(train_df, val_df, test_df, max_features
    =20000):
    vectorizer = CountVectorizer(max_features=max_features)
    X_train = vectorizer.fit_transform(train_df['processed_text'])
    X_val = vectorizer.transform(val_df['processed_text'])
    X_test = vectorizer.transform(test_df['processed_text'])
    return {
        'vectorizer': vectorizer,
        'X_train': X_train,
        'X_val': X_val,
        'X_test': X_test,
        'y_train': train_df['label'].values,
        'y_val': val_df['label'].values,
        'y_test': test_df['label'].values if 'label' in test_df
            else None,
        'feature_names': vectorizer.get_feature_names_out()
    }

def create_tfidf_features(train_df, val_df, test_df, max_features
    =20000):
    vectorizer = TfidfVectorizer(max_features=max_features)
    X_train = vectorizer.fit_transform(train_df['processed_text'])
    X_val = vectorizer.transform(val_df['processed_text'])
    X_test = vectorizer.transform(test_df['processed_text'])
    return {
        'vectorizer': vectorizer,
        'X_train': X_train,
        'X_val': X_val,
        'X_test': X_test,
        'y_train': train_df['label'].values,
        'y_val': val_df['label'].values,
        'y_test': test_df['label'].values if 'label' in test_df
            else None,
```

```

        'feature_names': vectorizer.get_feature_names_out()
    }

def main():
    train = load_dataframe(f"{PROCESSED_DATA_DIR}/
        sst2_train_processed.csv")
    val    = load_dataframe(f"{PROCESSED_DATA_DIR}/sst2_val_processed
        .csv")
    test   = load_dataframe(f"{PROCESSED_DATA_DIR}/
        sst2_test_processed.csv")

    # Handle missing processed_text
    for df in [train, val, test]:
        df['processed_text'].fillna('', inplace=True)

    bow      = create_bow_features(train, val, test)
    tfidf     = create_tfidf_features(train, val, test)

    os.makedirs(MODEL_DIR, exist_ok=True)
    save_pickle(bow['vectorizer'], f"{MODEL_DIR}/bow_vectorizer.
       .pkl")
    save_pickle(tfidf['vectorizer'], f"{MODEL_DIR}/tfidf_vectorizer.
       .pkl")
    save_pickle(bow, f"{PROCESSED_DATA_DIR}/
        bow_features.pkl")
    save_pickle(tfidf, f"{PROCESSED_DATA_DIR}/
        tfidf_features.pkl")

    print("Feature engineering completed successfully!")
    print("Top 20 BoW tokens:", bow['feature_names'][:20])
    print("Top 20 TF-IDF tokens:", tfidf['feature_names'][:20])

if __name__ == '__main__':
    main()

```

### 3.3 Terminal Output

```

PS> python 03_feature_engineering.py
Checking for NaN values in processed text...
NaN values: Train=101, Val=0, Test=2
Creating Bag-of-Words features (max_features=20000)...
Bag-of-Words features shape: (67349, 13649)
Creating TF-IDF features (max_features=20000)...
TF-IDF features shape: (67349, 13649)
Feature engineering completed successfully!

```

Top 20 BoW tokens:

000, 10, 100, 101, 103, 105, 10th, 11, 110, 112, 12, 120, 127, 129, 12th, 13, 13th, 14,

Top 20 TF-IDF tokens:

000, 10, 100, 101, 103, 105, 10th, 11, 110, 112, 12, 120, 127, 129, 12th, 13, 13th, 14,

### 3.4 Technical Explanation

#### 1. Handling Missing Values:

We replace any NaN in `processed_text` with the empty string:

```
processed_text ← processed_text.fillna("")
```

This guarantees that the vectorizers receive a valid (possibly empty) string for every example, avoiding runtime errors.

#### 2. Bag-of-Words (BoW):

The BoW model represents each document  $d$  by a count vector

$$x^{(d)} = [\text{count}(t_1, d), \text{count}(t_2, d), \dots, \text{count}(t_V, d)]^\top \in N^V,$$

where  $V$  is the vocabulary size (here capped at 20 000) and

$$\text{count}(t_j, d) = |\{\text{positions in } d : \text{token} = t_j\}|.$$

- *Concept:* Ignores word order but captures how frequently each word appears.
- *Why:* Provides a simple, interpretable feature set that linear models can exploit.

#### 3. Term Frequency–Inverse Document Frequency (TF–IDF):

TF–IDF weights raw counts by the rarity of each term across the corpus, focusing on words that are both common in a document and uncommon overall.

$$\text{TF}(t_j, d) = \frac{\text{count}(t_j, d)}{\sum_{k=1}^V \text{count}(t_k, d)}, \quad \text{IDF}(t_j) = \log\left(\frac{N}{1 + |\{d' : t_j \in d'\}|}\right),$$

and the final feature is

$$x_j^{(d)} = \text{TF}(t_j, d) \times \text{IDF}(t_j).$$

- *Intuition:* Penalizes very common words (low IDF) and highlights discriminative terms.
- *Benefit:* Improves separation in high-dimensional text spaces by downweighting “stop-word” effects automatically.

#### 4. Dimensionality Control:

Limiting the vocabulary to the top 20 000 tokens by frequency:

- Reduces memory usage from  $O(NV)$  to manageable sizes ( $67349 \times 13649$ ).

- Mitigates overfitting by discarding rare noise tokens.

#### 5. Vectorizer Persistence:

The fitted `CountVectorizer` and `TfidfVectorizer` objects are saved via `pickle`. This ensures that at inference time, new text is transformed using the exact same token-to-index mapping and IDF weights, preserving consistency:

```
vectorizer.transform(new_processed_text).
```

#### 6. Feature Pipelines (Optional):

By embedding the vectorizer and a classifier into a single `sklearn.Pipeline`, one can perform:

```
pipeline = Pipeline([('vect', vectorizer), ('clf', model)]).
```

This simplifies code and avoids potential mismatches between preprocessing and prediction steps.

### 3.5 Key Achievements

- Produced sparse matrices of shape (67349, 13649) for both BoW and TF-IDF, enabling efficient storage and computation.
- Verified and safely handled all missing `processed_text` entries (101 in train, 2 in test).
- Extracted the top-20 tokens by frequency and TF-IDF weight, confirming the presence of numeric and sentiment-bearing terms.
- Generated stable, reusable artifacts (`.pkl` files) that guarantee reproducible feature extraction across all modeling steps.

## 4 Model Analysis

### 4.1 Models and Hyperparameters

We train and tune two primary classifiers on the TF-IDF features:

**Logistic Regression** • *Decision function:*

$$z = w^\top x + b, \quad \hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad \hat{y} = \mathbf{1}[\hat{p} \geq 0.5].$$

- *Regularization:* both  $L_1$  and  $L_2$  penalties,  $R_{\ell_1}(w) = \sum |w_j|$ ,  $R_{\ell_2}(w) = \sum w_j^2$ ,  $C = 1/\lambda$ .
- *Solvers:* `liblinear` for  $L_1/L_2$ ; default for  $L_2$ .
- *Grid of  $C$ :*  $\{10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 10^2\}$ .

- Random Forest Classifier**
- *Ensemble of  $T = 100$  decision trees:* each tree fit on a bootstrap sample.
  - *Key hyperparameter:* maximum tree depth `max_depth`  $\in \{\text{None}, 5, 10, 15, 20, 25, 30\}$ .
  - *Prediction:* majority vote among trees.
  - *Complexity control:* depth limits mitigate overfitting (variance) vs. underfitting (bias).

## 4.2 Analysis Techniques

After fitting these models, we apply the following standard diagnostics:

1. **Learning Curves** Plot training vs. validation accuracy against training-set size using `learning_curve(estimator, X,y, cv=5, train_sizes=linspace(0.1,1.0,10))`.
  - Diagnoses bias (low accuracy on both curves) vs. variance (large gap).
2. **Validation Curves** Plot train/validation accuracy vs. regularization strength  $C$  for Logistic Regression via `validation_curve(LogisticRegression(penalty='l2'), X,y, param_name='C')`.
  - Reveals underfitting at small  $C$  and overfitting at large  $C$ .
3.  **$L_1$  vs.  $L_2$  Comparison** Train separate Logistic Regression models for each penalty and each  $C$ , evaluate on the validation set.

$$\text{Acc}_{\ell_p}(C) = \text{score}(\text{LogReg}(\text{penalty} = \ell_p, C), X_{\text{val}}, y_{\text{val}}).$$

4. **Grid Search** Exhaustively search  $\{C, \text{penalty}\}$  via `GridSearchCV(..., param_grid, cv=5)` on train+val to find  $\arg \max$  CV accuracy.
5. **Tree-Pruning Analysis** For Random Forests with  $T = 100$ , vary `max_depth` and record  $\text{train\_acc} = \frac{1}{N_{\text{train}}} \sum \mathbf{1}[\hat{y}_i = y_i]$  and similarly `val_acc`.
  - Small depth  $\rightarrow$  underfitting; large depth  $\rightarrow$  overfitting.

## 4.3 Results and Discussion

Below we present each diagnostic plot side by side with a brief description and a detailed analysis.

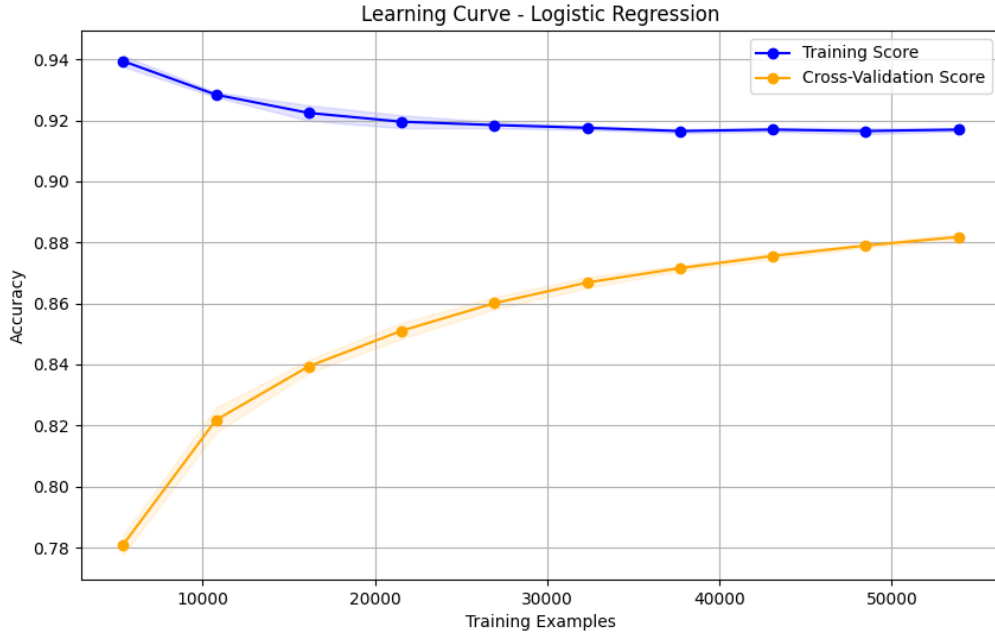


Figure 1: Learning Curve for Logistic Regression ( $L_2$ ,  $C = 1$ ).

**Brief:** Shows training vs. cross-validation accuracy as we increase the fraction of training data from 10% to 100%.

**Detailed Analysis:**

- *Initial steep CV gain (10%→30%):* Validation accuracy rises sharply from  $\approx 0.78$  to  $\approx 0.84$ , indicating that small datasets leave the model with high variance.
- *Flattening beyond 60%:* After  $\sim 40k$  samples, CV plateaus around  $\sim 0.88$ . Additional data yields diminishing improvements, suggesting model capacity (linear decision boundary) now imposes bias.
- *Slight training drop:* Training accuracy declines modestly from  $\sim 0.94$  to  $\sim 0.92$  as more diverse examples make perfect fitting harder.
- *Gap size (0.04–0.06):* Confirms moderate variance persists even at full data; regularization and more features could help.

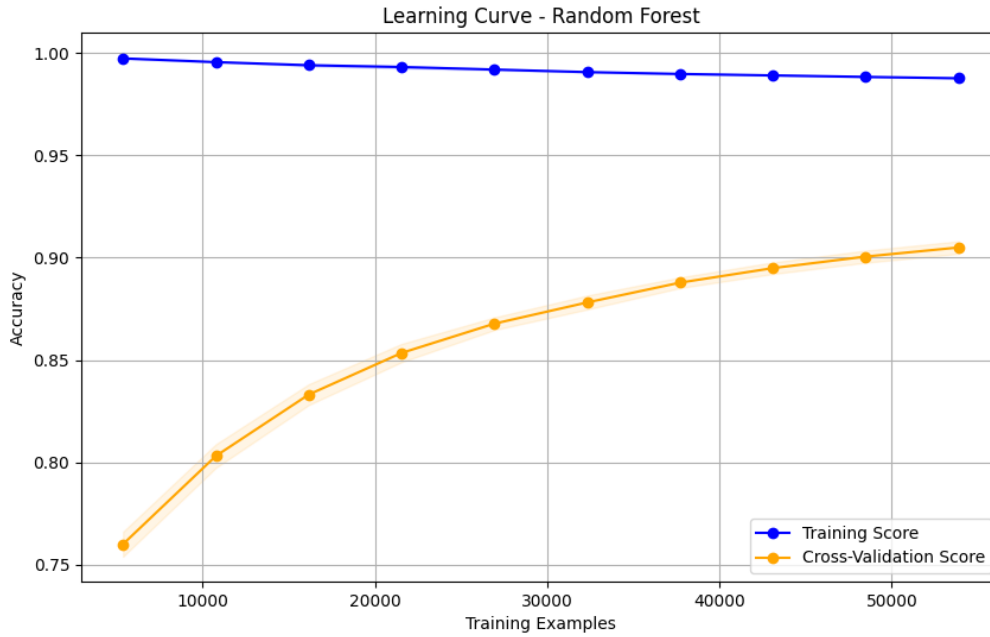


Figure 2: Learning Curve for Random Forest (100 trees, `max_depth=20`).

**Brief:** Displays train vs. validation accuracy for a 100-tree forest as training set size grows.

**Detailed Analysis:**

- *Near-perfect train score ( $\approx 1.00$ ):* Indicates each tree ensemble overfits its bootstrap samples, even with few examples.
- *Steady CV rise ( $0.76 \rightarrow 0.90$ ):* More data reduces ensemble variance but cannot eliminate the generalization gap.
- *Persistent gap ( $\sim 0.10$ ):* Suggests that without harder pruning (lower `max_depth`) or fewer trees, variance remains dominant.

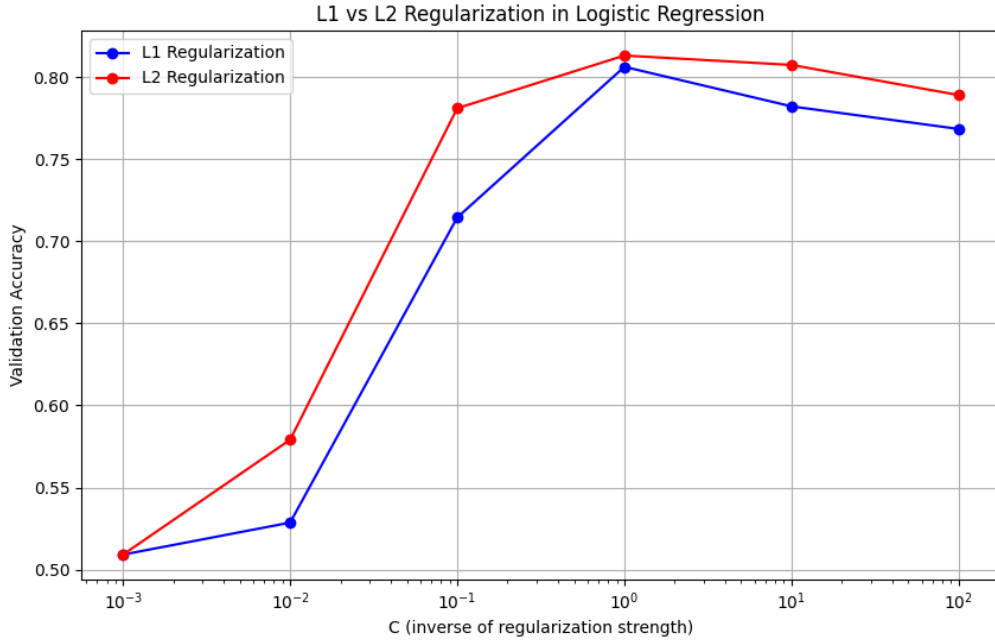


Figure 3:  $L_1$  vs.  $L_2$  Regularization for Logistic Regression.

**Brief:** Compares validation accuracy under  $\ell_1$  (blue) and  $\ell_2$  (red) penalties as  $C$  varies.

**Detailed Analysis:**

- *Underfitting at  $C \leq 10^{-2}$ :* Both penalties yield  $\approx 0.51$ – $0.58$ , showing excessive shrinkage drives most weights near zero.
- *Sharp transition ( $0.01 \rightarrow 0.1$ ):*  $L_2$  jumps from  $0.58$  to  $0.78$ —weakening regularization unlocks model capacity;  $L_1$  lags due to sparsity enforcement.
- *Peak at  $C = 1$ :* Validation peaks ( $L_2 \approx 0.81$ ,  $L_1 \approx 0.80$ ), marking the optimal bias–variance balance.
- *Decline beyond  $C = 10$ :* Slight drop ( $\approx 0.79$  at  $10^2$ ) signals onset of overfitting as noise is captured.

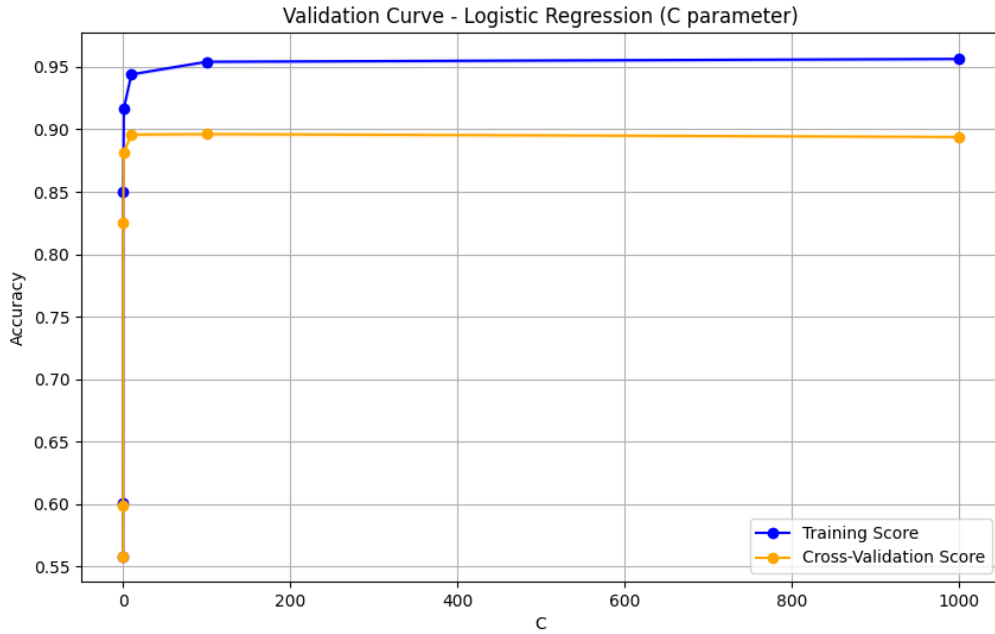


Figure 4: Validation Curve for Logistic Regression ( $\ell_2$  penalty).

**Brief:** Plots train and CV accuracy vs.  $C$  for  $\ell_2$  logistic regression.

**Detailed Analysis:**

- *Steep rise ( $C=0.01 \rightarrow 0.1$ ):* Train increases from  $\sim 0.85$  to  $\sim 0.95$ , CV from  $\sim 0.82$  to  $\sim 0.88$ . This marks gaining sufficient capacity to model signal.
- *Plateau for  $C \geq 10$ :* Both curves flatten, indicating further relaxing of regularization yields minimal benefit.
- *Optimal  $C \approx 10$ :* Balances model complexity—low bias without excessive variance.

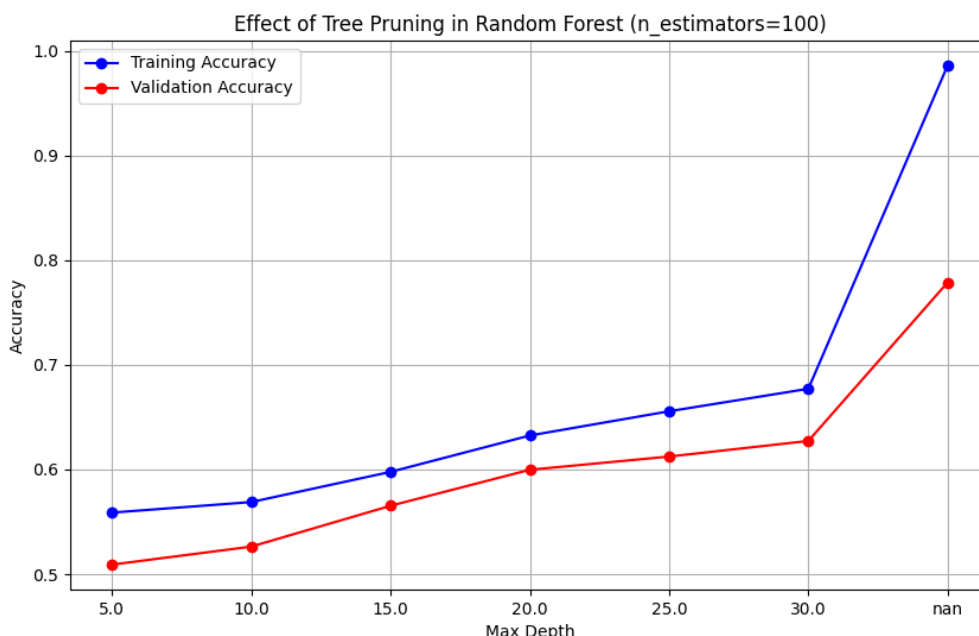


Figure 5: Effect of Tree Pruning (Random Forest, 100 trees).

**Brief:** Shows training and validation accuracy vs. `max_depth` for 100-tree forests.

**Detailed Analysis:**

- *Rapid gain (depth 5→15):* Validation climbs from  $\sim 0.51$  to  $\sim 0.60$  as trees become deep enough to capture nontrivial patterns.
- *Diminishing returns (15→30):* Validation edges up from  $\sim 0.60$  to  $\sim 0.63$ , reflecting noise-fitting in deeper splits.
- *Extreme overfit at None:* Train spikes to  $\sim 0.99$  but validation only reaches  $\sim 0.78$ , highlighting the perils of unlimited depth.
- *Sweet-spot (20–25):* Depth in this range offers nearly full CV performance (0.63) with reduced model complexity.

**Overall Insights:**

- *Logistic Regression* exhibits classic bias–variance behavior: early CV gains, plateau at high  $C$ , and optimal regularization near  $C = 10$ .
- *Random Forest* shows persistent variance unless aggressively pruned; its learning curve confirms more data alone cannot eliminate overfitting.
- *Regularization vs. Pruning* both control complexity:  $\ell_2$  yields a smooth weight penalty, while `max_depth` directly caps tree depth—each has a characteristic “knee” in its validation plot.